

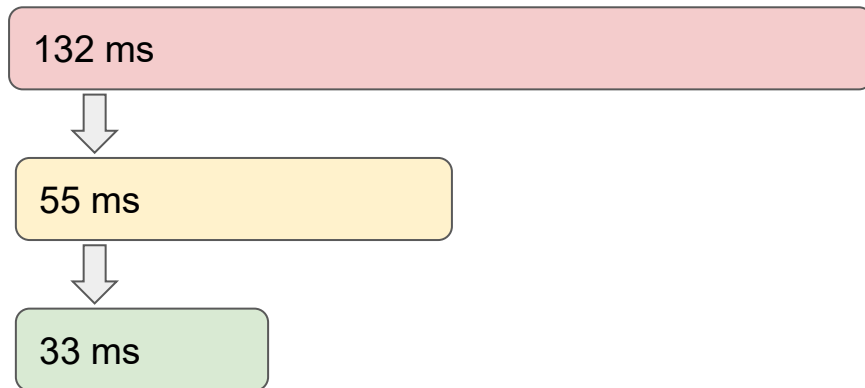
# 16,6 мс на кадр - а главный поток один

Бюджет кадра при 60 FPS – 16,6 мс.

Ядер в машине – 8-16, но по умолчанию работает один main thread: ввод, симуляция, подготовка рендера.

Остальные ядра в это время простаивают. Железо есть – архитектуры, которая им пользуется, нет.

## Кейс Naughty Dog TLOU Remastered



Портирование The Last of Us на PS4 под 60 FPS. Рецепт — “jobify everything”: систему за системой переносили на fiber-based job system, пока кадр не уехал со 132 до 33 мс.

# Мягкое реальное время: два класса работы

## В бюджет кадра

дедлайн - 16,6 мс; промах = dropped frame

- обработка ввода
- симуляция и геймплей
- подготовка и submit рендера
- всё, от чего зависит этот кадр

Здесь параллелим: ту же работу делим между ядрами, чтобы уложиться в дедлайн.

## Живёт фоном

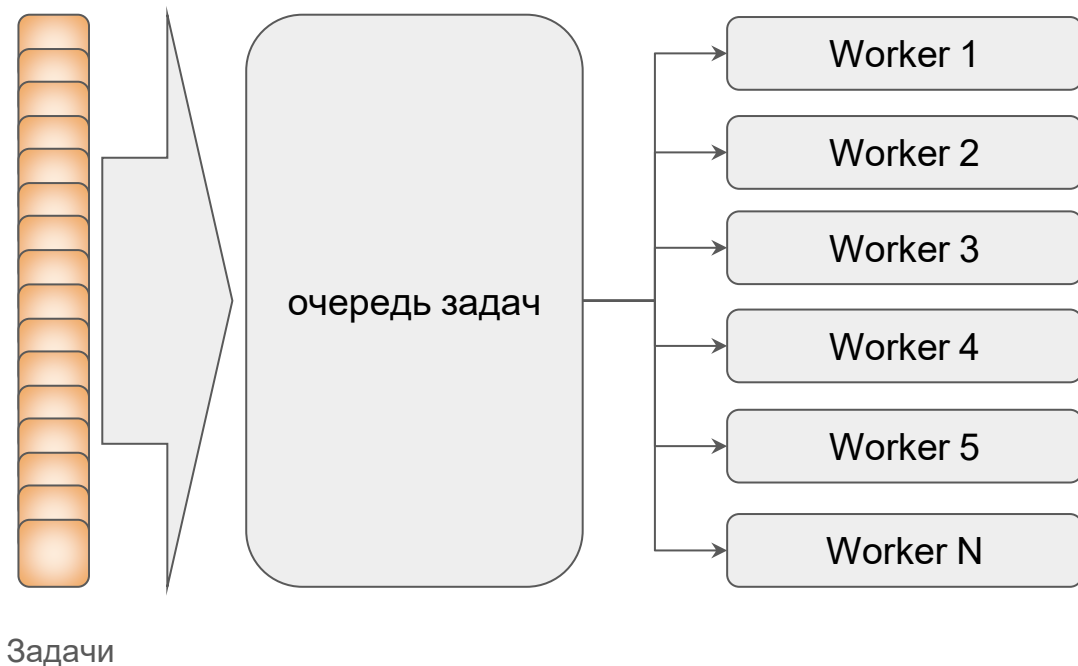
дедлайн – “к тому моменту, как понадобится”

- загрузка и парсинг ресурсов
- стриминг уровня
- декодирование текстур и звука
- телеметрия

Здесь выносим: убираем работу с main thread, чтобы он не стоял на месте.

Job system обслуживает оба класса: первую – распараллеливает, вторую – выносит.  
В ЛР 1 вы сделаете и то, и другое.

# Инверсия: работа ищет свободные потоки



Task – единица работы: функция + данные, без привязки к потоку.

Потоков ровно столько, сколько ядер. Main thread – тоже участник, а не диспетчер.

Планировщик – ваш код, а не ОС: вы решаете, что и в каком порядке выполняется.

Масштабирование = число ядер, а не число подсистем.

# Анатомия минимальной системы: pool + очередь + workers

## Потокобезопасная очередь задач

Task = std::function или своя структура; push – из любого потока, pop – worker'ами.

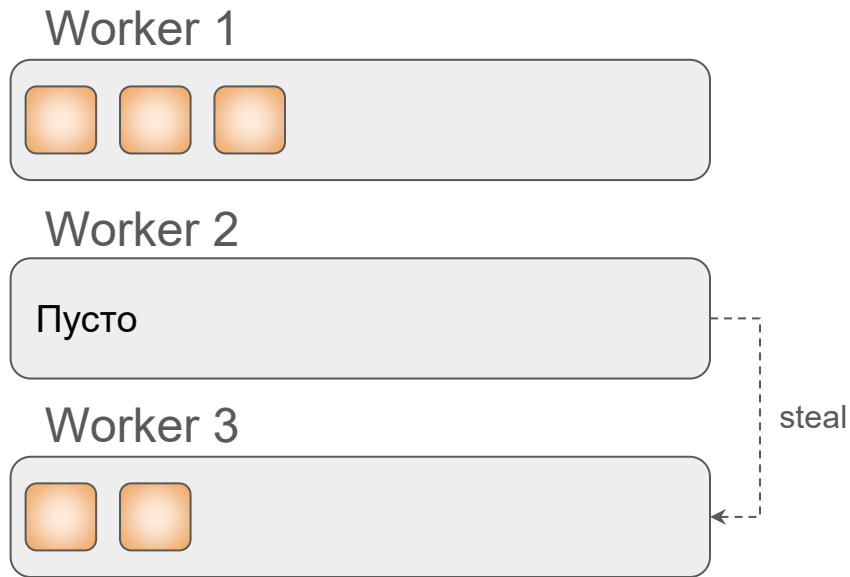
N workers = cores – 1, цикл: pop → execute

Запускаются один раз при инициализации и живут всю сессию – никаких thread-per-task.

condition\_variable будит спящих worker'ов

Producer-consumer; ничего нового, кроме масштаба применения.

# Work stealing: очередь на поток вместо одной общей



Одна общая очередь = точка contention на каждой постановке и каждом взятии задачи.

Per-thread очереди: worker кладёт и берёт локально – без конкуренции в общем случае.

Освободившийся worker **крадёт задачу из чужой очереди** – жертва выбирается случайно или по кругу.

Балансировка нагрузки получается сама собой: у кого очередь длиннее, у того и крадут.

Алгоритм: Blumofe, Leiserson, [Scheduling Multithreaded Computations by Work Stealing](#); та же схема – в Wicked

# Waiting thread helps: Wait – это не сон

```
void Wait(const context& ctx)
{
    if (IsBusy(ctx))
    {
        PriorityResources& res =
internal_state.resources[int(ctx.priority)];

        // Wake any threads that might be sleeping:
        res.sleepingCondition.notify_all();

        // work() will pick up any jobs that are on standby
and execute them on this thread:
        res.work(res.next_queue_index());

        while (IsBusy(ctx)) { /* доспать на waitingCondition
                                condvar */ }
    }
}
```

Интуиция подсказывает: Wait = заснуть до завершения.

На деле: ждущий поток будит спящих, сам забирает задачи из очереди и засыпает, только когда работы не осталось вообще.

Следствие: вложенный Wait не блокирует пул — задача может дожждаться подзадач, не отнимая поток.

# Весь API – на одном экране

```
void Initialize(uint32_t maxThreadCount = ~0u);
void ShutDown();

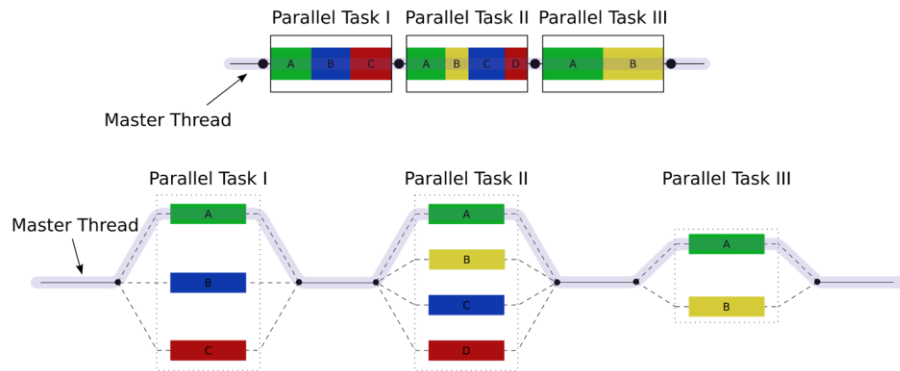
uint32_t GetThreadCount(Priority priority = Priority::High);

// Add a task to execute asynchronously. Any idle thread will
// execute this.
void Execute(context& ctx, const job_function_type& task);

// Divide a task onto multiple jobs and execute in parallel.
void Dispatch(context& ctx, uint32_t jobCount, uint32_t groupSize,
const job_function_type& task, size_t sharedmemory_size = 0);

// Check if any threads are working currently or not
bool IsBusy(const context& ctx);

// Wait until all threads become idle
// Current thread will become a worker thread, executing jobs
void Wait(const context& ctx);
```



# context – атомарный счётчик, а не future

```
enum class Priority
{
    High,          // Default
    Low,           // Pool of low priority threads, useful
                  // for generic tasks that shouldn't interfere with high
                  // priority tasks
    Streaming,     // Single low priority thread, for
                  // streaming resources
    Count
};

// Defines a state of execution, can be waited on
struct context
{
    std::atomic<uint32_t> counter{ 0 };
    Priority priority = Priority::High;
};
```

**Почему не future/promise:** future – это shared state и аллокация на каждый вызов; здесь – один atomic на весь workload.

context создаётся на стеке; priority в том же struct – выбор пула бесплатно при постановке.

А чем плох std::async

Непрозрачный глобальный пул: на одной платформе – общий с чужими задачами, на другой – свежий поток на вызов. Ни там ни там нет **лимита параллелизма**, **приоритетов** (загрузка уровня важнее мипов) и **единого шатдауна**.



# Три пула по приоритетам – готовый паттерн под загрузку

Приоритеты вшиты и в размеры пулов, и в планировщик ОС

```
// Calculate the actual number of worker threads we want:
switch (priority)
{
case Priority::High:
    res.numThreads = internal_state.numCores - 1; // -1 for
main thread
    break;
case Priority::Low:
    res.numThreads = internal_state.numCores - 2; // -1 for
main thread, -1 for streaming
    break;
case Priority::Streaming:
    res.numThreads = 1;
    break;
}
```

# JobQueue: блоки по 256 задач и early exit без лока

```
struct JobQueue
{
    struct Block
    {
        uint32_t first_item = 0;
        uint32_t last_item = 0;
        Block* next = nullptr;
        Job items[256];
    };
    wi::allocator::BlockAllocator<Block, 16> allocator;
    Block* first_block = nullptr;
    Block* last_block = nullptr;
    std::mutex locker;
    //wi::SpinLock locker;
    std::atomic_uint32_t cnt{ 0 }; // for early exit, reduce
    contention on locker in job stealing scenario

    JobQueue() { first_block = last_block = allocator.allocate(); }

    __forceinline void push_back(const Job& item);
    __forceinline bool pop_front(Job& item);
};
```

Блоки по 256 из BlockAllocator:  
память переиспользуется, нет  
malloc на задачу.

Атомарный cnt: пустая очередь  
определяется без захвата  
мьютекса → stealing не  
упирается в лок чужой очереди.

# Dispatch: семантика compute-шейдера на CPU

```
struct JobArgs
{
    uint32_t jobIndex;      // job index relative to dispatch (like SV_DispatchThreadID in HLSL)
    uint32_t groupID;       // group index relative to dispatch (like SV_GroupID in HLSL)
    uint32_t groupIndex;    // job index relative to group (like SV_GroupIndex in HLSL)
    bool isFirstJobInGroup; // is the current job the first one in the group?
    bool isLastJobInGroup;  // is the current job the last one in the group?
    void* sharedmemory;     // stack memory shared within the current group (jobs within a group
                           // execute serially)
};

//-----
Job::execute():
for (uint32_t j = groupJobOffset; j < groupJobEnd; ++j)
{
    args.jobIndex = j;
    args.groupIndex = j - groupJobOffset;
    args.isFirstJobInGroup = (j == groupJobOffset);
    args.isLastJobInGroup = (j == groupJobEnd - 1);
    task(args);
}
```

**jobCount / groupSize:**

дешёвые задачи пакуются в группы – амортизация накладных расходов.

**shared memory** – аллока на стеке потока, живая в пределах группы: частичные суммы, промежуточные буферы.

Нарочито как в HLSL compute: переносите интуицию с GPU – и обратно

# Детали ремесла: affinity, выравнивание, 96 байт

```
int core = threadID + 1; // put threads on increasing cores starting from 2nd
DWORD_PTR affinity_result = SetThreadAffinityMask(handle, affinityMask);
```

Каждый worker прибит к своему ядру – нет миграций и прогревов кэша. Streaming – на последнее ядро:  $core = numCores - 1$ .

```
struct alignas(64) Job { ... }
```

Задача занимает свою кэш-линию и не делит её с соседями — защита от false sharing при stealing.

```
#ifdef JOB_SYSTEM_FIXED_SIZE_FUNCTION
    using job_function_type = wi::function<void(JobArgs), 96>; // 96 is
    chosen to fit the whole job storage with parameters in 128 bytes
#else
    using job_function_type = std::function<void(JobArgs)>;
#endif // JOB_SYSTEM_FIXED_SIZE_FUNCTION
```

Свой fixed-size function вместо `std::function` – без скрытых аллокаций при постановке задачи.

# Dispatch по компонентам: параллельный update ECS

```
void Scene::RunCameraUpdateSystem(wi::jobsystem::context& ctx)
{
    wi::jobsystem::Dispatch(ctx, (uint32_t)cameras.GetCount(),
        small_subtask_groupsize, [&](wi::jobsystem::JobArgs args) {

        CameraComponent& camera = cameras[args.jobIndex];
        Entity entity = cameras.GetEntity(args.jobIndex);
        const TransformComponent* transform =
        transforms.GetComponent(entity);
        if (transform != nullptr)
        {
            camera.TransformCamera(*transform);
        }
        camera.UpdateCamera();
    });
}
```

Это штатный способ работы в живом движении, а не учебный пример.

Ваша задача №2 «снимок → диапазоны» — ровно этот паттерн, даже поверх хеш-таблиц.

Чтобы эффект был измерим, нужна сцена с тысячами сущностей — сделайте “замерочную”.

# Фоновая загрузка и многопоточный Render()

Загрузка в фоне: [LoadingScreen](#)

Цепочка: LoadModel → Archive (раскладка байтов .wiscene, read/write, FIFO) → Scene::Serialize — тяжёлые компоненты порождают подзадачи (seri.ctx).

Имена текстур из компонентов → resource manager: кэш по имени, чтение и декод на streaming-потоке.

RenderPath::Load() — пустой hook: тело пишет игра; LoadingScreen исполняет его как job (в репо зовётся из Lua: AddLoadModelTask), прогресс — из счётчика задач.

Кадр не стоит на месте — это и есть критерий L1 из вашего ТЗ, только уже в продакшн-движке.

Render() — const и без потока в контракте

[запись команд идёт в несколько потоков через job system](#)

Const-контракт = read-only фаза: такой же read/write-анализ вы будете делать в задаче №2.

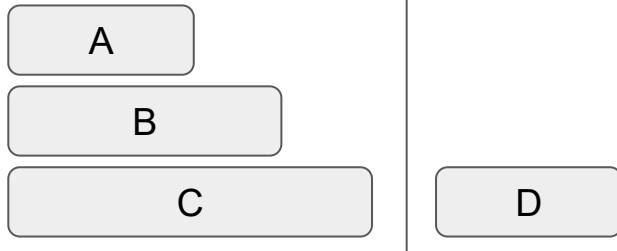
# Предел модели: fork-join ставит лишние барьеры

В модели Wicked зависимости выражаются вложенными **Dispatch + Wait**.

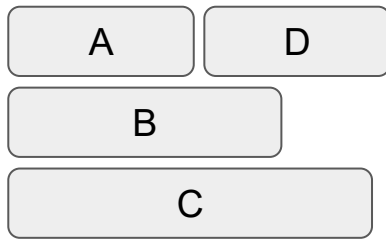
Но Wait ждёт весь workload — все задачи разом, а не те, что реально нужны следующему шагу.

Итог: барьеры там, где работа могла бы перекрываться. D зависит только от A, но стартует, когда закончатся A, B и C.

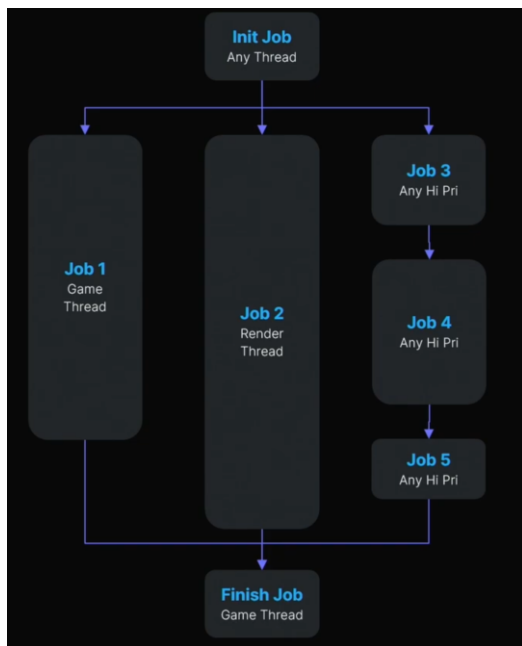
fork-join



fork-join



# Task graph: зависимости – это рёбра, а не Wait



```
FGraphEventRef InitJob = FFunctionGraphTask::CreateAndDispatchWhenReady([]  
{  
    // InitJob, Any Thread, Runs immediately  
}, TStatId(), nullptr, ENamedThreads::AnyThread);  
  
FGraphEventRef Job1 = FFunctionGraphTask::CreateAndDispatchWhenReady([]  
{  
    // Job 1, Game Thread, Runs After InitJob  
}, TStatId(), InitJob, ENamedThreads::GameThread);  
  
FGraphEventRef Job2 = FFunctionGraphTask::CreateAndDispatchWhenReady([]  
{  
    // Job 2, Render Thread, Runs After InitJob  
}, TStatId(), InitJob, ENamedThreads::GameThread);  
  
FGraphEventRef PreviousJob = InitJob;  
for (int32 JobIdx = 3; JobIdx < 6; ++JobIdx)  
{  
    // Depend on previously run job  
    PreviousJob = FFunctionGraphTask::CreateAndDispatchWhenReady([]  
    {  
        // Job, Any Hi Pri, Runs After PreviousJob  
    }, TStatId(), PreviousJob, ENamedThreads::AnyHiPriThreadNormalTask);  
}  
  
FGraphEventArray Prerequisites = { Job1, Job2, PreviousJob };  
FFunctionGraphTask::CreateAndDispatchWhenReady([]  
{  
    // Finish Job, GameThread, Runs After Everything else  
}, TStatId(), &Prerequisites, ENamedThreads::GameThread);
```

[How to Benefit from Multithreading in Your Unreal Engine Projects](#)



# UE TaskGraph: named threads и prerequisites

FGraphEvent – хэндл задачи-узла: на него можно подписаться как на prerequisite.

Prerequisites – явные зависимости между событиями: задача стартует, когда закрыты все входы.

Named threads: часть работы прибита к потокам – Game / Render / RHI. У GPU-API и игрового состояния есть владельцы.

Связь: граф задаёт порядок, named thread – место. Ребра свободно пересекают границы потоков – планировщик организует передачу.

Это не абстракция: тики акторов – такие же узлы (FTickFunctionTask) с prerequisites. Схема справа упрощена: реальный кадр – сотни узлов.

